

Mock Design: A Distributed File Storage Service

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

1. Concept From First Principles: Applying the Framework

Distributed file storage (Dropbox/Google Drive-style) is a different problem shape than nearly everything else in this Phase: instead of small structured records (a message, an order, a post), the core payload is large, unstructured binary data (files, potentially gigabytes each) that must be uploaded efficiently, stored durably, and kept in sync across multiple devices for the same user — introducing chunking and sync-conflict resolution, two genuinely new concepts for this program.

Step 1 — Clarify requirements

- Functional: upload/download files; organize files into folders; sync changes automatically across a user's multiple devices; support file versioning (recover a previous version).
- Non-functional: files must never be lost or corrupted (durability is paramount); uploads/downloads of large files should be efficient (not re-transferring an entire file for a small edit); sync should propagate to other devices within seconds.

Step 2 — Estimate scale

Assume 50 million users, each storing an average of 5GB, giving 250 petabytes of total stored data — a genuinely enormous storage footprint that immediately rules out any single-machine or even small-cluster storage design, and firmly establishes that the storage layer itself must be a purpose-built, horizontally-scalable object store, not a general-purpose database.

Step 3 — API

POST /files/upload {fileMetadata} -> {uploadUrl, fileId} (chunk-aware, detailed in Step 5), GET /files/{id}, GET /sync?deviceId=X&since=timestamp -> [changedFiles...] for a device to pull recent changes since its last sync.

Step 4 — High-level design

Deliberately separate two concerns that are easy to conflate: a **Metadata Service** (file names, folder structure, version history, sharing permissions — small, structured, relational-shaped data, a good fit for a traditional database) and a **Block/Object Storage Service** (the actual file bytes — large, unstructured, a good fit for purpose-built object storage like S3 or a self-hosted equivalent, deliberately not the same database as metadata). A **Sync Service** notifies a user's other connected devices when a change occurs, using a persistent connection per device (Phase 3, Ch 9's real-time delivery techniques, directly reused from Chapter 7's chat design) rather than requiring devices to poll.

Step 5 — Deep dive: chunking, dedup, and sync conflict resolution

Chunking: rather than treating a file as one indivisible blob, split it into fixed-size chunks (e.g., 4MB each) and hash each chunk (SHA-256). This unlocks two real benefits worth naming explicitly: (1) when a user edits a large file, only the changed chunks need re-uploading, not the entire file — a significant

bandwidth saving for, say, a 2GB video file with a small edit; (2) **deduplication across users** becomes possible — if a chunk's hash already exists in storage (perhaps another user uploaded an identical file, or the same file was uploaded twice), the upload can skip re-storing that chunk entirely and just reference the existing one, since content-identical chunks are, by definition, indistinguishable regardless of which user or file they came from.

Sync conflict resolution: the genuinely hard, most-probed part of this design. If a user edits the same file on two devices while offline, then both devices come back online and try to sync, which version wins? A naive "last write wins" approach silently discards one user's work, which is a poor and sometimes damaging outcome for a file-storage product. The standard, more defensible answer: on detecting a genuine conflict (both devices modified the same file since their last common synced version), **preserve both** — keep the server's version as the canonical file, and save the losing/conflicting device's version as a separate "conflicted copy" file, letting the user manually reconcile rather than silently losing data. Detecting the conflict in the first place typically uses a version vector or a simple version-number-plus-last-synced-version comparison per file, so the system can distinguish a genuine concurrent edit from a normal sequential update.

Step 6 — Trade-offs and wrap-up

Volunteer: cross-user chunk deduplication raises a real privacy/security consideration worth naming — deduplicating based purely on content hash could theoretically let an attacker infer whether a specific known file exists elsewhere in the system by checking if a chunk hash is already stored, so production systems typically encrypt chunks with a key that limits cross-user dedup benefits, or scope deduplication only within a single user's own files — a genuine security/efficiency trade-off, not a purely technical one.

2. Real-World Example / Case Study

Dropbox's own widely-discussed engineering blog posts about their chunking and delta-sync approach, and Google Drive's equivalent block-based storage architecture, are the canonical real-world instances of exactly this design — Dropbox in particular has published detailed accounts of moving away from third-party cloud storage to their own purpose-built block storage system specifically to optimize this chunking and dedup behavior at their scale.

3. Diagram

```

Upload flow (chunked):
[Client] -> split file into 4MB chunks -> hash each (SHA-256)
      |
      for each chunk: does hash already exist in storage?
          yes -> skip upload, reference existing chunk (dedup)
          no  -> upload chunk to [Object Storage]
      |
      [Metadata Service]: record file -> ordered list of chunk hashes

Sync conflict detection:
Device A edits file (v3 -> v4)      Device B edits file (v3 -> v4')
both sync after being offline -----> CONFLICT detected
(both diverged from common ancestor v3)
      |
      keep server version as canonical
      save other device's version as "conflicted copy" file

```

4. Trade-offs Table

Decision	Alternative	Why this choice
Separate Metadata Service and Object Storage	One combined database for everything	Structured metadata and large binary blobs have very different storage needs
Chunked upload with content-hash dedup	Upload entire file every time	Saves bandwidth on edits; enables cross-file/cross-user deduplication
Preserve both versions on sync conflict	Last-write-wins	Avoids silently discarding a user's work; lets the user manually reconcile
Persistent connection for sync notification	Devices poll periodically for changes	Faster propagation (seconds, not poll-interval delay), reuses Ch 7/Ph3 Ch9 pattern

5. Common Interview Questions

Q1: "Why split files into chunks instead of storing them as one blob?"

Model approach: Name both benefits explicitly — only changed chunks need re-uploading on edit, and content-hash-based deduplication becomes possible across files and users, both meaningful savings at this scale.

Q2: "Two devices edit the same file while offline. How do you resolve the conflict when they both sync?"

Model approach: Reject last-write-wins explicitly, and describe preserving both versions (canonical server version plus a conflicted-copy file) as the standard, data-loss-avoiding resolution strategy.

Q3: "What's a security concern with content-based chunk deduplication across users?"

Model approach: Name the file-existence inference risk directly — an attacker could probe whether a known file exists elsewhere by checking chunk-hash presence — and the encryption/dedup-scoping

mitigations that address it.

6. Practice Exercises

- 1 Run the full six-step framework for "design a file storage and sync service like Dropbox," out loud, with a timer.
- 2 Design the metadata schema for a file's chunk list, including how you'd represent an edited file that shares most chunks with its previous version.
- 3 Design the version-vector or version-number comparison logic that detects a genuine sync conflict versus a normal sequential update.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Design Google Drive or Dropbox (Cloud File Sharing Service) System Design Interview Prep"	Directly covers chunking, metadata separation, and sync for this exact problem.
English	"Design file-sharing system like Google Drive / Dropbox (System design interview with EM)"	Full mock interview covering conflict resolution in depth.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering storage systems and sync as part of the broader curriculum.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Hindi roadmap reinforcing object storage and metadata separation fundamentals.

8. Key Takeaways

- File storage introduces a genuinely new payload shape for this program: large, unstructured binary data, requiring chunking rather than whole-file transfer.
- Separating metadata (structured, small) from object storage (unstructured, large) is the foundational architectural split for this problem.
- Content-hash-based chunking enables both bandwidth-efficient partial re-uploads and cross-file/cross-user deduplication.
- Sync conflict resolution should preserve both versions rather than silently discarding one via last-write-wins.
- Version vectors or version-number comparisons are what let the system distinguish a genuine concurrent conflict from a normal sequential edit.
- Cross-user deduplication raises a real, nameable security trade-off around file-existence inference.
- Persistent connections for sync notification directly reuse Chapter 7's and Phase 3 Chapter 9's real-time delivery patterns.